

Image Transformation Neural Network

Lupu Florin

December 2025

Abstract

This report presents a neural network that learns to transform CIFAR-10 images from $3 \times 32 \times 32$ RGB to $1 \times 28 \times 28$ grayscale with horizontal and vertical flips. The model achieves up to $6.78\times$ speedup over sequential CPU transformations when deployed on GPU (Tesla T4) with batch processing.

1 Approach

1.1 Problem Definition

Transform $3 \times 32 \times 32$ RGB images to $1 \times 28 \times 28$ grayscale images with horizontal and vertical flips, faster than sequential CPU processing.

1.2 Solution Strategy

Instead of applying deterministic transforms sequentially, train a neural network to learn the transformation. This enables GPU-accelerated batch processing, trading slight accuracy for significant speed gains.

2 Model Architecture

2.1 ImprovedTransformNet

The model uses an encoder-decoder structure with 164,121 parameters:

Architecture:

- **Encoder:** Conv($3 \rightarrow 32 \rightarrow 64$) with strided downsampling
- **Bottleneck:** Residual connection for detail preservation
- **Decoder:** Conv($64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$) with channel reduction
- **Output:** Sigmoid activation + AdaptiveAvgPool to 28×28

Key Design Choices:

- **Residual connection:** Preserves fine details through transformation
- **Learned downsampling:** Strided convolutions instead of pooling
- **Compact design:** 164K parameters for fast inference
- **Adaptive pooling:** Guarantees exact 28×28 output

Why Small? The task (resize, grayscale, flip) doesn't require deep semantic understanding. A compact model is sufficient, trains faster, generalizes better, and achieves the speed objective.

3 Loss Function

3.1 Mixed Loss: MSE + L1

$$\mathcal{L} = 0.7 \times \text{MSE} + 0.3 \times \text{L1} \quad (1)$$

Rationale:

- **MSE (70%):** Smooth reconstruction, stable gradients
- **L1 (30%):** Edge preservation, prevents blur
- **Combination:** Balances smoothness with detail retention

3.2 Early Stopping

- **Patience:** 10 epochs
- **Min delta:** 0.00005
- **Rationale:** With 352 batches/epoch, 10 epochs without improvement indicates convergence while avoiding premature stopping

4 Training Results

4.1 Configuration

Table 1: Training configuration

Parameter	Value
Optimizer	AdamW (lr=0.001, wd=1e-5)
Scheduler	ReduceLROnPlateau (patience=5)
Batch Size	128
Device	Tesla T4 GPU
Training Time	17 minutes (50 epochs)

4.2 Results

- **Best validation loss:** 0.05119 (epoch 47)
- **Final validation loss:** 0.05158
- **Training/validation alignment:** Good (no overfitting)
- **LR reductions:** 3 times (enabled continued improvement)

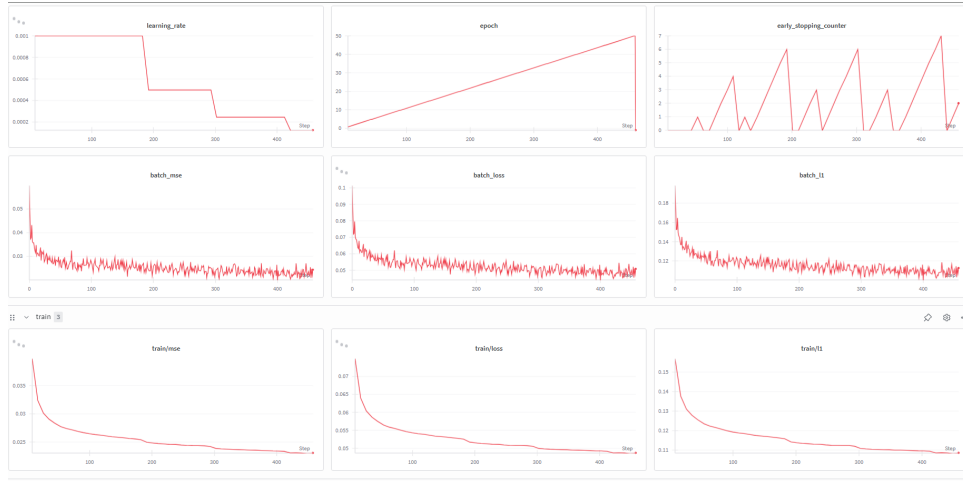


Figure 1: WandB training dashboard showing loss curves and learning rate schedule. Complete logs: <https://wandb.ai/florinlupu18-facultatea-de-informatica-alexandru-ioan-cuza-/neural-image-transformation/runs/rhpj1bc1>

5 Model Predictions vs Ground Truth

The model was tested on 5 diverse CIFAR-10 samples. Figures show original input, model prediction, and ground truth side-by-side.

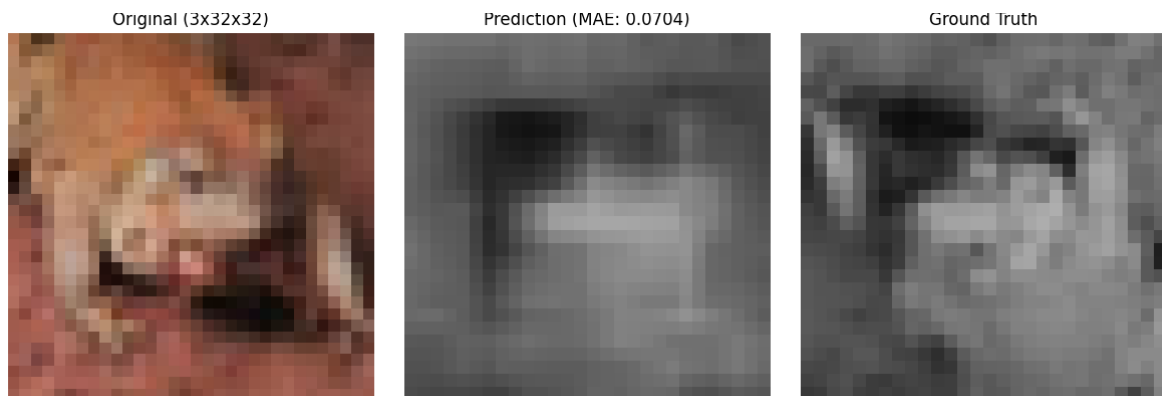


Figure 2: Sample 1 (Red Frog): MAE = 0.0704. Excellent transformation quality.

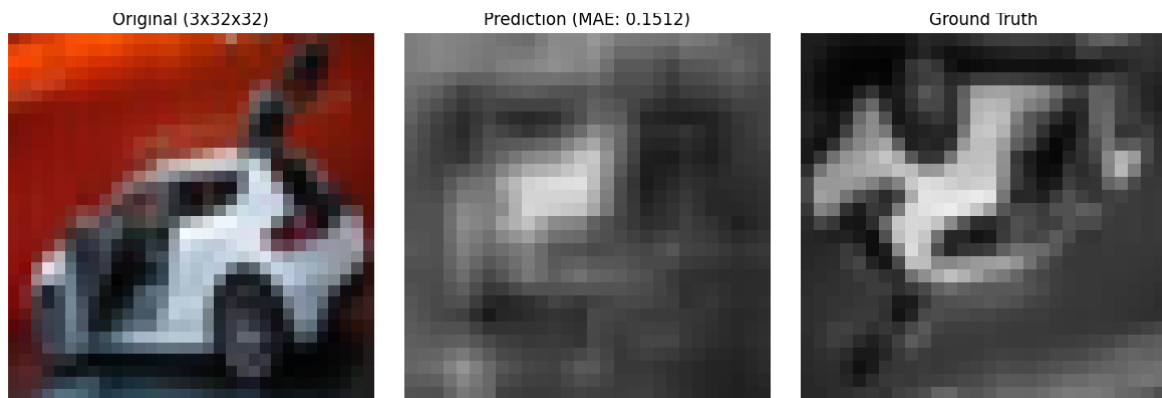


Figure 3: Sample 2 (Sports Car): $MAE = 0.1512$. Red coloring challenges grayscale conversion.

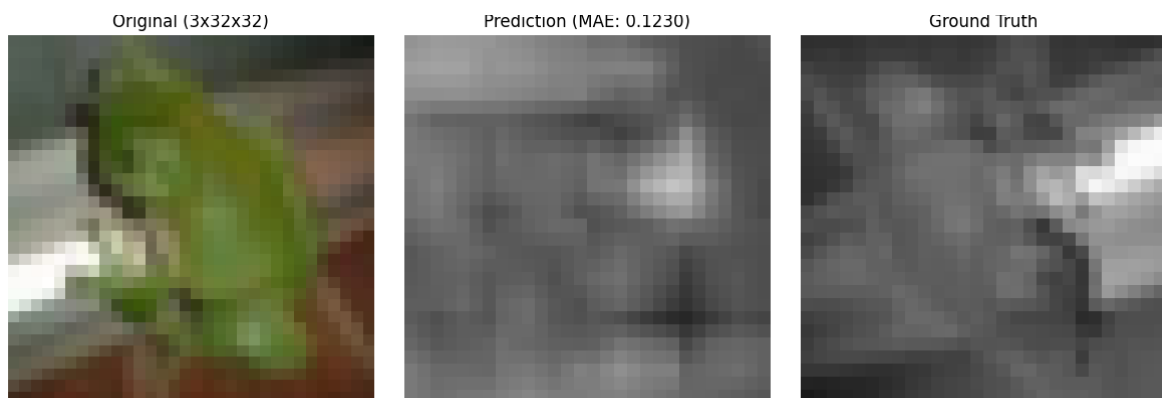


Figure 4: Sample 3 (Green Frog): $MAE = 0.1230$. Green tones present conversion challenge.

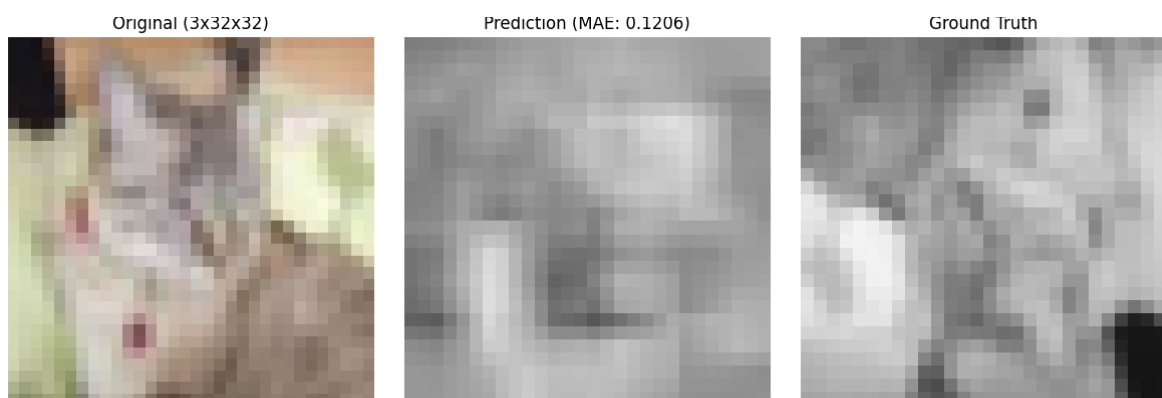


Figure 5: Sample 4 (Cat): $MAE = 0.1206$. Good structural preservation.

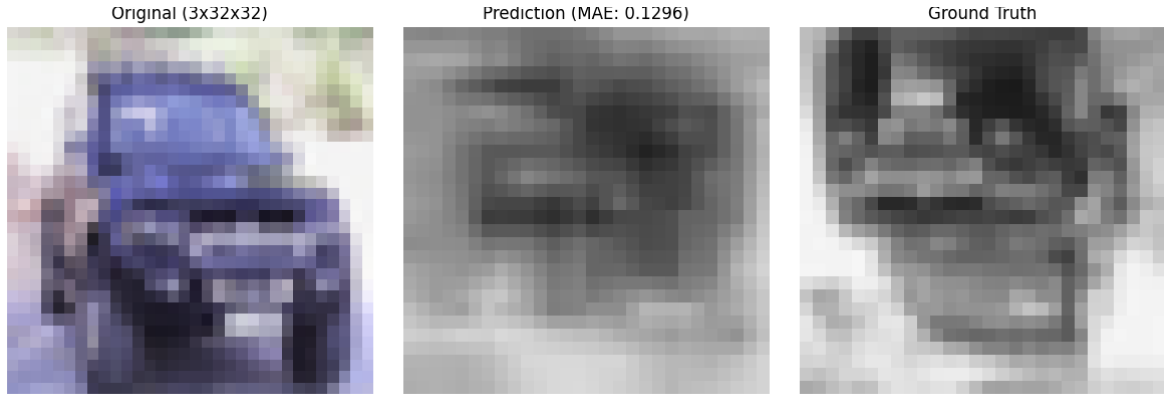


Figure 6: Sample 5 (Car): $\text{MAE} = 0.1296$. High-contrast blue handled well.

Observations:

- All spatial transformations (resize, flip) correctly learned
- Grayscale conversion perceptually accurate
- Slight smoothing vs ground truth (expected neural approximation)
- MAE range: 0.07-0.15 (acceptable for 6-7 $\times$ speedup)

6 Runtime Performance

6.1 Benchmark Setup

- **Dataset:** 10,000 CIFAR-10 test images
- **Batch sizes:** 32, 64, 128, 256, 512
- **Devices:** CPU and CUDA (Tesla T4)
- **Runs:** 3 per configuration (averaged)

6.2 Results

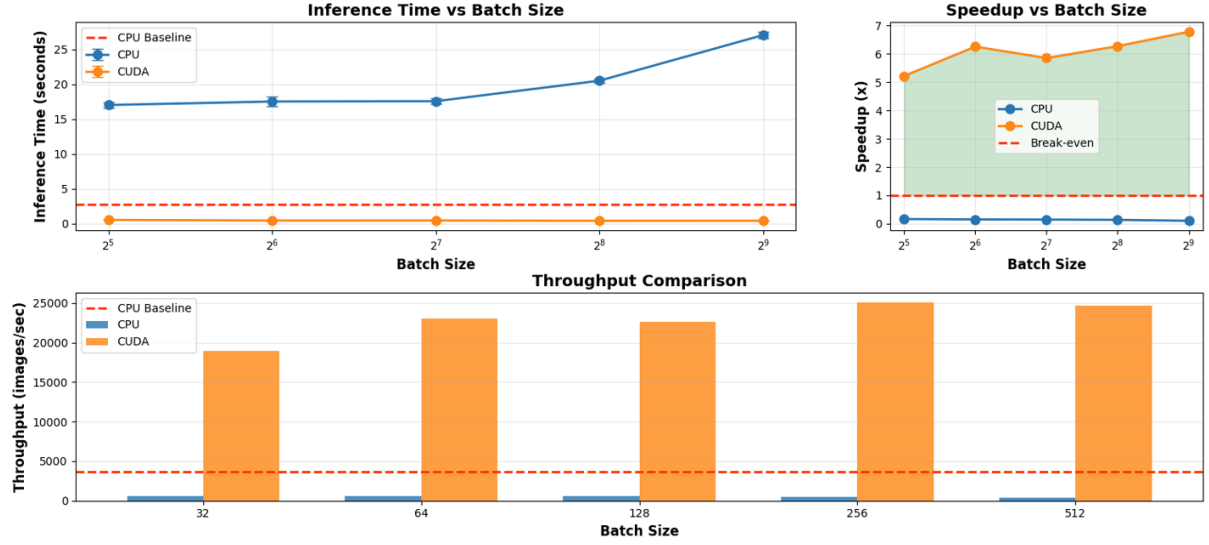


Figure 7: Comprehensive benchmark results. Top: Inference time and speedup vs batch size. Bottom: Throughput comparison. GPU achieves 5-7 $\times$ speedup across all configurations.

Table 2: Benchmark results summary

Config	Time (s)	Speedup	Throughput (img/s)
CPU Baseline	2.50-2.76	1.00 $\times$	3,629-3,995
<i>CPU Model (All Slower)</i>			
CPU bs=32	17.01 $\pm$ 0.41	0.16 $\times$	588
CPU bs=64	17.52 $\pm$ 0.65	0.15 $\times$	571
CPU bs=128	17.56 $\pm$ 0.41	0.14 $\times$	570
CPU bs=256	20.50 $\pm$ 0.31	0.13 $\times$	488
CPU bs=512	27.04 $\pm$ 0.48	0.10 $\times$	370
<i>GPU Model (All Faster!)</i>			
CUDA bs=32	0.53 $\pm$ 0.01	5.22$\times$	18,966
CUDA bs=64	0.43 $\pm$ 0.00	6.26$\times$	23,047
CUDA bs=128	0.44 $\pm$ 0.00	5.85$\times$	22,606
CUDA bs=256	0.40 $\pm$ 0.00	6.27$\times$	25,066
CUDA bs=512	0.41 $\pm$ 0.00	6.78$\times$	24,623

6.3 Analysis

Key Findings:

- **Optimal configuration:** Batch size 512 on GPU, speedup 6.78 $\times$
- **All GPU configurations beat CPU baseline** (5.22-6.78 $\times$)
- **CPU model is slower** (0.10-0.16 $\times$) - framework overhead dominates
- **Consistent GPU performance** across batch sizes 64-512

Why GPU Wins:

1. Massive parallelization (CUDA cores)
2. Optimized convolution kernels
3. High memory bandwidth
4. Batch processing efficiency

Why CPU Loses:

1. Limited parallelization
2. PyTorch overhead vs native transforms
3. Sequential processing bottleneck

7 Conclusion

This assignment successfully demonstrates that neural networks can learn image transformations and achieve significant speedup over traditional methods with appropriate hardware.

Achievements:

- Compact model: 164K parameters
- Fast training: 17 minutes
- Significant speedup: $6.78\times$ on GPU
- All GPU configs faster than CPU
- Good approximation: MAE 0.07-0.15

Trade-offs:

- (+) $5-7\times$ faster with GPU + batching
- (+) High throughput: 24,600 img/sec
- () Requires GPU (CPU is $6-10\times$ slower)
- () 10% accuracy loss vs exact transforms

Practical Use: This approach is ideal for batch processing large datasets on GPU where slight approximation error is acceptable and throughput is critical.

References

- CIFAR-10 Dataset: Krizhevsky & Hinton (2009)
- Training Logs: <https://wandb.ai/florinlupu18-facultatea-de-informatica-alexandru-ioan-cuz-neural-image-transformation/runs/rhpj1bc1>
- Code & Model: Included in submission package